

```
//SIT 55 : ASSIGNMENT 1 : SEARCHING AND SORTING
/*Consider a student database of SEIT class (at least 15 records).
Database contains different fields of
every student like Roll No, Name and SGPA.(array of structure)
a) Design a roll call list, arrange list of students according to roll
numbers in ascending order (Use
Bubble Sort)
b) Arrange list of students alphabetically. (Use Insertion sort)
c) Arrange list of students to find out first ten toppers from a class.
(Use Quick sort)
d) Search students according to SGPA. If more than one student having
same SGPA, then print list
of all students having same SGPA.
e) Search a particular student according to name using binary search
without recursion. (all the
student records having the presence of search key should be displayed)
(Note: Implement either Bubble sort or Insertion Sort.)*/
```

```
#include<iostream>
#include<string.h>
using namespace std;
int const size=5;
struct student
{
    int roll;
    char name[20];
    float SGPA;
};
class Student_Data
{
public:
    void accept(struct student list[size]);
    void display(struct student list[size]);
    void bubbleSort(struct student list[size]);
    void InsertionSort(struct student list[size]);
    void LinearSearch(struct student list[size]);
    void BinarySearch(struct student list[size]);
    void QuickSort(struct student list[size],int low, int high);
    int partition(struct student list[size],int low, int high);
};
void Student_Data::accept(struct student list[size])
{
    int i,j=1;
    for(i=0;i<size;i++){
        cout<<"\nENTER STUDENT "<<j<<" ROLL NO. :-";
        cin>>list[i].roll;
        cout<<"\nEnter Student "<<j<<" NAME :-";
        cin>>list[i].name;
        cout<<"\nEnter Student "<<j<<" SGPA :-";
        cin>>list[i].SGPA;
        j++;
    }
}
void Student_Data::display(struct student list[size])
{
    int i;
    for(i=0;i<size;i++){
        cout<<"\n*****";
        cout<<"\nSTUDENT ROLL NO. :- "<<list[i].roll;
        cout<<"\nSTUDENT NAME :- "<<list[i].name;
```

```

        cout<<"\nSTUDENT SGPA :- "<<list[i].SGPA;
        cout<<"\n*****";
    }

}

void Student_Data::bubbleSort(struct student list[size])
{
    int i,j;
    struct student temp;
    for(i=0;i<size-1;i++){
        for(j=0;j<(size-1-i);j++){
            if(list[j].roll>list[j+1].roll){
                temp=list[j];
                list[j]=list[j+1];
                list[j+1]=temp;
            }
        }
    }
}

void Student_Data::InsertionSort(struct student list[size])
{
    int k,j;
    struct student temp;
    for(k=1;k<size;k++){
        temp=list[k];
        j=k-1;

        while (strcmp(list[j].name,temp.name)> 0 && j>=0){
            list[j+1]=list[j];
            j--;
        }
        list[j+1]=temp;

    }
}

void Student_Data::LinearSearch(struct student list[size])
{
    float SGPA;
    int i;
    cout<<"\n ENTER SGPA: ";
    cin>>SGPA;
    for(i=0;i<size;i++){
        if(SGPA==list[i].SGPA){
            cout<<"\n*****";
            cout<<"\nSTUDENT ROLL NO. :- "<<list[i].roll;
            cout<<"\nSTUDENT NAME :- "<<list[i].name;
            cout<<"\nSTUDENT SGPA :- "<<list[i].SGPA;
            cout<<"\n*****";
        }
    }
}

void Student_Data::BinarySearch(struct student list[size])
{
    int low, high, mid;
    char search[80];
    cout<<"\n ENTER NAME OF STUDENT YOU WANT TO SEARCH: ";
    cin>>search;
    low=0;
    high=size-1;
    while(low<=high)

```

```

{
mid=(low+high)/2;

if(strcmp(list[mid].name,search)==0)
{
cout<<"\n*****";
cout<<"\nSTUDENT ROLL NO. :- "<<list[mid].roll;
cout<<"\nSTUDENT NAME :- "<<list[mid].name;
cout<<"\nSTUDENT SGPA :- "<<list[mid].SGPA;
cout<<"\n*****";
break;
}
else
{
if(strcmp(list[mid].name,search)<0)
{
low=mid+1;
}
else
{
high=mid-1;
}
}
}
if(strcmp(list[mid].name,search)!=0)
{
cout<<"\n***DATA WITH NAME "<<search<<"NOT FOUND*****\n";
}

}
int Student_Data::partition(struct student list[size],int low, int high)
{
float pivot = list[high].SGPA;
int i=low-1;
for(int j=low;j<high;j++)
{
if(list[j].SGPA > pivot)
{
struct student temp;
i++;
temp=list[i];
list[i]=list[j];
list[j]=temp;
}
}
struct student temp;
i++;
temp=list[i];
list[i]=list[high];
list[high]=temp;
return i;
}
void Student_Data::QuickSort(struct student list[size],int low, int high)
{
if(low<high)
{
int pi=partition(list,low,high);
QuickSort(list, low, pi-1);
QuickSort(list, pi+1, high);
}
}

```

```

}
int main(){
    int ch,i;
    bool a=true;
    struct student data[size];
    Student_Data sd;
    sd.accept(data);
    cout<<"*****INSERTED DATA*****\n\n";
    sd.display(data);
    cout<<"*****\n\n";

    while (a)
    {
        cout<<"MENU:\n 1. Bubble Sort (Sort List By Roll number) \n 2. Insertion
Sort (Sort List by Name) \n 3. Linear Search \n 4. Binary Search\n 5.
Quick Sort\n";
        cout<<"\nEnter Choice :- ";
        cin>>ch;
        cout<<endl;
        switch (ch)
        {
            case 1:
                sd.bubbleSort(data);
                sd.display(data);
                break;
            case 2:
                sd.InsertionSort(data);
                sd.display(data);
                break;
            case 3:
                sd.LinearSearch(data);
                break;
            case 4:
                sd.InsertionSort(data);
                sd.BinarySearch(data);
                break;
            case 5:
                sd.QuickSort(data,0,size-1);

                cout<<"\n***** First 3 Topper Students *****";
                for(int i=0;i<3;i++){
                    cout<<"\nSTUDENT ROLL NO. :- "<<data[i].roll;
                    cout<<"\nSTUDENT NAME :- "<<data[i].name;
                    cout<<"\nSTUDENT SGPA :- "<<data[i].SGPA;
                    cout<<"\n*****";
                }

                break;

            default:
                cout<<"\n\nENTER VALID CHOICE !";
                break;
        }
        cout<<"\nDO YOU WANT TO PERFORM MORE OPERATIONS?(1/0)\n";
        cin>>a;
    }
    return 0;
}

```

/*OUTPUT:

ENTER STUDENT 1 ROLL NO. :-5

Enter Student 1 NAME :-p

Enter Student 1 SGPA :-9

ENTER STUDENT 2 ROLL NO. :-2

Enter Student 2 NAME :-a

Enter Student 2 SGPA :-9.6

ENTER STUDENT 3 ROLL NO. :-6

Enter Student 3 NAME :-m

Enter Student 3 SGPA :-7

ENTER STUDENT 4 ROLL NO. :-9

Enter Student 4 NAME :-j

Enter Student 4 SGPA :-8

*****INSERTED DATA*****

STUDENT ROLL NO. :- 5

STUDENT NAME :- p

STUDENT SGPA :- 9

STUDENT ROLL NO. :- 2

STUDENT NAME :- a

STUDENT SGPA :- 9.6

STUDENT ROLL NO. :- 6

STUDENT NAME :- m

STUDENT SGPA :- 7

STUDENT ROLL NO. :- 9

STUDENT NAME :- j

STUDENT SGPA :- 8

MENU:

1. Bubble Sort (Sort List By Roll number)
2. Insertion Sort (Sort List by Name)
3. Linear Search
4. Binary Search
5. Quick Sort

Enter Choice :- 1

```
STUDENT ROLL NO. :- 2
STUDENT NAME :- a
STUDENT SGPA :- 9.6
*****
*****
STUDENT ROLL NO. :- 5
STUDENT NAME :- p
STUDENT SGPA :- 9
*****
*****
STUDENT ROLL NO. :- 6
STUDENT NAME :- m
STUDENT SGPA :- 7
*****
*****
STUDENT ROLL NO. :- 9
STUDENT NAME :- j
STUDENT SGPA :- 8
*****
DO YOU WANT TO PERFORM MORE OPERATIONS?(1/0)
1
MENU:
1. Bubble Sort (Sort List By Roll number)
2. Insertion Sort (Sort List by Name)
3. Linear Search
4. Binary Search
5. Quick Sort
```

Enter Choice :- 2

```
*****
STUDENT ROLL NO. :- 2
STUDENT NAME :- a
STUDENT SGPA :- 9.6
*****
*****
STUDENT ROLL NO. :- 9
STUDENT NAME :- j
STUDENT SGPA :- 8
*****
*****
STUDENT ROLL NO. :- 6
STUDENT NAME :- m
STUDENT SGPA :- 7
*****
*****
STUDENT ROLL NO. :- 5
STUDENT NAME :- p
STUDENT SGPA :- 9
*****
DO YOU WANT TO PERFORM MORE OPERATIONS?(1/0)
1
MENU:
1. Bubble Sort (Sort List By Roll number)
2. Insertion Sort (Sort List by Name)
3. Linear Search
4. Binary Search
5. Quick Sort
```

Enter Choice :- 3

ENTER SGPA: 7

STUDENT ROLL NO. :- 6

STUDENT NAME :- m

STUDENT SGPA :- 7

DO YOU WANT TO PERFORM MORE OPERATIONS?(1/0)

1

MENU:

1. Bubble Sort (Sort List By Roll number)
2. Insertion Sort (Sort List by Name)
3. Linear Search
4. Binary Search
5. Quick Sort

Enter Choice :- 4

ENTER NAME OF STUDENT YOU WANT TO SEARCH: a

STUDENT ROLL NO. :- 2

STUDENT NAME :- a

STUDENT SGPA :- 9.6

DO YOU WANT TO PERFORM MORE OPERATIONS?(1/0)

1

MENU:

1. Bubble Sort (Sort List By Roll number)
2. Insertion Sort (Sort List by Name)
3. Linear Search
4. Binary Search
5. Quick Sort

Enter Choice :- 5

***** First 3 Topper Students *****

STUDENT ROLL NO. :- 2

STUDENT NAME :- a

STUDENT SGPA :- 9.6

STUDENT ROLL NO. :- 5

STUDENT NAME :- p

STUDENT SGPA :- 9

STUDENT ROLL NO. :- 9

STUDENT NAME :- j

STUDENT SGPA :- 8

DO YOU WANT TO PERFORM MORE OPERATIONS?(1/0)

0*/